

CONFIDENTIAL SECURITY ASSESSMENT

Swaptics - Current Main Branch

Full-stack security, database, authentication and release-readiness audit

SECURITY SCORE	PRODUCTION READINESS	OVERALL RISK
6 / 10	6.5 / 10	HIGH

Branch: main | Commit: 646790e (Updated security)

Production build, tests and lint were executed against the audited source.

Assessment date: 18 August 2026

1. Executive Summary

The current branch is materially safer than the previously audited versions. Swaptics has returned to a single Supabase Auth model, aligning browser sessions with `auth.uid()` and the database RLS policies. The anonymous administrator bootstrap, Firebase bridge, cross-project Firebase token verifier and merge-conflict release blockers are no longer present in the current source.

The production build succeeds and 28 tests pass. However, the branch should not yet be considered fully production-ready. Two older permissive RLS policies remain active and are combined with newer policies using PostgreSQL's permissive OR behavior. As a result, any authenticated user can still create arbitrary notification records and seller badges. The user-deletion path is also described as atomic but the UI invokes a sequential Edge Function instead of the transactional database function.

No live exploitation was performed. The review used static source analysis and safe local verification. A current dependency advisory scan was not performed because it would transmit dependency metadata to an external service without separate approval.

Verification summary

Check	Result	Notes
Git state	PASS	main matches origin/main; only audit output/tmp untracked
Production build	PASS	2,897 modules transformed; route splitting present
Tests	PASS	28/28 tests across two files
Lint	FAIL	4 errors and 94 warnings
Dependency advisories	NOT RUN	External metadata-sharing approval not granted

Risk summary

Severity	Count	Release effect
Critical	0	No confirmed critical source finding
High	4	Resolve before production launch
Medium / Low	10	Hardening, reliability and quality backlog

2. High-Risk Findings

1. Permissive notification policy remains active

HIGH

Impact: Any authenticated user can insert a notification for any user, allowing spam, impersonated system messages, phishing prompts and notification flooding.

Evidence: Migration 20260223191724 creates 'Authenticated can insert notifications' WITH CHECK (true). The new hardening migration drops only 'System can insert notifications', so the permissive policy remains and is OR-combined with the new policy.

Recommended fix: Explicitly drop the old policy and replace it with a policy that binds the recipient and allowed type, or create notifications only through a validated server function.

2. Any authenticated user can grant seller badges

HIGH

Impact: Users can award arbitrary trust badges to themselves or others, corrupting marketplace trust and enabling fraud.

Evidence: Migration 20260223191724 creates 'Authenticated can insert badges' WITH CHECK (true). The hardening migration drops only 'System can insert badges'; the older policy remains active alongside the admin-only policy.

Recommended fix: DROP POLICY IF EXISTS 'Authenticated can insert badges' and permit service-role or verified-admin insertion only. Add a database trigger or audited server workflow.

3. User deletion is not atomic despite security claims

HIGH

Impact: A failure midway can leave partial deletion, orphaned records or an account whose data was removed while authentication remains active.

Evidence: The migration defines transactional `admin_delete_user_cascade()`, but `Admin.tsx` invokes the `delete-user` Edge Function. That function performs sequential service-role deletes, ignores most returned errors, and treats auth deletion failure as a warning.

Recommended fix: Invoke the transactional RPC from a server-verified endpoint, check its result, delete storage separately with retry tracking, and do not report success until authentication deletion completes.

4. Legacy OTP endpoint remains an abuse surface

HIGH

Impact: An unauthenticated caller can generate outbound email cost and exercise service-role account-creation paths. Email-only throttling is easy to distribute across addresses.

Evidence: `send-otp` uses the service role, sends email, and exposes `send/verify` actions without CAPTCHA, IP/device throttling or a global budget. The main UI no longer needs this flow after moving back to Supabase Auth.

Recommended fix: Remove and undeploy the function if unused. Otherwise add gateway rate limits, CAPTCHA, generic responses, atomic attempt counters and operational abuse alerts.

3. Medium and Low Findings

5. Admin setup likely calls a nonexistent SDK method

setup-admin and send-otp call supabase.auth.admin.getUserByEmail, but that method is absent from the installed Supabase client API. The admin-add flow may always report user not found or fail at runtime. Use a supported server-side lookup strategy and test the deployed function.

6. Upload validation remains client-controlled

Size, MIME and extension checks are useful but browser-supplied values are not authoritative. Validate file signatures, dimensions and quotas server-side, re-encode images, and clean metadata.

7. CSP still permits unsafe-inline

unsafe-eval was removed, which is an improvement. Replace inline scripts/styles with nonces or hashes to strengthen XSS resistance.

8. Notification sender authorization is incomplete

Even after dropping the permissive policy, clients should not freely select recipient, title or message. Create notifications from server-side business events.

9. Product-view counts are easy to manipulate

The authenticated product_views INSERT policy uses WITH CHECK (true), allowing repeated artificial views. Add uniqueness/rate constraints or aggregate server-side.

10. Admin dashboard lacks pagination

It loads complete profiles, products, reports, views and chats, causing unnecessary private-data exposure and poor performance as the platform grows.

11. Security tests duplicate logic

The 27 security tests copy validation functions instead of importing production code and do not exercise RLS, Edge Functions or cross-user access. They can pass while production behavior is insecure.

12. Lint gate is not clean

Four errors and 94 warnings remain, including hook dependency issues and widespread any types. Keep lint blocking in CI and reduce warnings to a controlled baseline.

13. SEO remains SPA-dependent

Dynamic social metadata may not be visible to link-preview bots. Add prerendering/SSR, Product and Article structured data, and dynamic sitemap entries.

14. Dependency advisory status is unknown

The audit did not submit package metadata to an external advisory service. Run an approved dependency scan in CI and retain the report as release evidence.

4. Improvements Since the Previous Audit

- Authentication is consolidated on Supabase Auth; Firebase bridge code and Firebase token-verification risk are removed.
- Anonymous administrator bootstrap and public role-table access are removed from current migrations.
- Role management now requires an authenticated administrator.
- Setup-admin requires an authenticated existing administrator and no longer accepts passwords or creates users.
- Image uploads enforce client-side size, MIME and extension allowlists.
- Database constraints enforce positive prices, selling price not exceeding original price, image count and nonempty names.
- Chat insertion is awaited and failed optimistic messages are rolled back.
- CSP removed unsafe-eval and added form-action plus upgrade-insecure-requests.
- Production build succeeds with route-level chunks and no oversized initial application bundle warning.
- Security-focused validation tests were added; all 28 tests pass.

Important migration note

PostgreSQL permissive RLS policies are combined using OR. Adding a stricter policy does not override an older permissive policy. Security migrations must explicitly drop every older policy that grants overlapping access, then verify the effective policy set in the deployed database.

5. Access Required to Complete Remediation

Use individual MFA-protected accounts and grant the minimum role needed. Do not share passwords, recovery codes, service-role keys or database credentials through chat.

System	Minimum access	Purpose
Repository	Write access to a remediation branch and CI	Policy migrations, Edge Function fixes, tests and lint
Supabase project	Developer; temporary Admin for secrets and production recovery	Inspect deployed policies, functions, auth, logs and storage
Database	Read-only production inspection, then migration-runner access	Confirm effective RLS policies and apply reviewed migrations
Vercel	Project Developer; Admin only for environment variables/domains	Preview deployment, CSP, logs and rollback
Email provider	Restricted API-key and abuse-dashboard access	Remove or harden OTP/email functionality
Monitoring	Read and alert-management access	Role changes, abuse, Edge Function and deletion failures
Staging	Admin test account plus two ordinary test users	Cross-user, RLS, admin and destructive-flow validation

Approvals and information required

- Confirmation that 20260817_security_hardening.sql was applied to production
- A read-only export of pg_policies for the audited tables
- Decision to delete or retain the legacy send-otp function
- Backup and rollback approval before RLS migration
- Authorization for staging-only cross-account security testing
- Explicit approval before running an external dependency advisory scan

6. Prioritized Action Plan

P0 - Policy correction

- Drop the old permissive notification and badge policies
- Inspect the deployed pg_policies state rather than relying only on migration files
- Add RLS regression tests for ordinary user, owner and administrator roles

P1 - Privileged workflow repair

- Route deletion through one transactional server operation
- Fix the unsupported email lookup in setup-admin
- Remove or harden send-otp
- Add immutable audit logs for role grants and deletions

P2 - Application hardening

- Add server-side image validation
- Make notification generation server-controlled
- Add view-count abuse controls
- Paginate and minimize admin queries

P3 - Release quality

- Resolve four lint errors and reduce warnings
- Replace copied security-test logic with production imports
- Add Supabase/RLS integration tests
- Run an approved dependency advisory scan

P4 - Defense in depth

- Remove unsafe-inline using CSP nonces or hashes
- Add monitoring and abuse budgets
- Complete SSR/prerendered SEO and structured data
- Conduct an authorized staging penetration test

7. Release Gate

A production release should require the following evidence:

- Effective production policies no longer include the permissive notification or badge INSERT rules
- Cross-user RLS tests demonstrate denial
- Admin role grants and deletion are verified end-to-end
- Production build, lint and tests pass in CI
- Legacy OTP endpoint is removed or protected by layered abuse controls
- Rollback and monitoring procedures are documented
- Dependency advisories are reviewed under an approved CI process

Conclusion

The branch has moved from critical architectural risk to a substantially healthier Supabase-native security model. The remaining high-risk issues are narrower and fixable, but the overlapping RLS policies and non-atomic deletion path must be resolved before Swaptics is treated as production-ready.

End of report